

Clase 264 — Pentest de aplicaciones iOS

Parte: **13 — Seguridad móvil, IoT e inalámbrica** · Fuente: OWASP MASTG y *The Mobile Application Hacker's Handbook* (Chell et al.) 🕒 Duración estimada: **120 min** · Nivel: **Avanzado**

🎯 Objetivo

Realizar una evaluación de seguridad de una app iOS sobre un dispositivo con jailbreak: descifrar y analizar el IPA, inspeccionar el almacenamiento y el Keychain, interceptar tráfico TLS evadiendo pinning, y saltar detección de jailbreak con Frida/objectection. El alumno aprenderá a adaptar la metodología MASVS al ecosistema iOS, donde el code signing y el sandbox imponen restricciones ausentes en Android.

⚠️ **Nota ética:** solo sobre apps propias, apps deliberadamente vulnerables (DVIA-v2, MASTG apps) o con autorización escrita. Descifrar o redistribuir apps de terceros vulnera la ley y los términos de Apple.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Descifrar** el binario Mach-O de una app propia con frida-ios-dump.
2. **Analizar** estáticamente el IPA (Info.plist, entitlements, cadenas, class-dump).
3. **Inspeccionar** el almacenamiento local y el Keychain con objection.
4. **Interceptar** tráfico TLS instalando un CA propio y evadiendo pinning.
5. **Evadir** detección de jailbreak en runtime con Frida.
6. **Reportar** hallazgos mapeados a OWASP MASVS con evidencia.

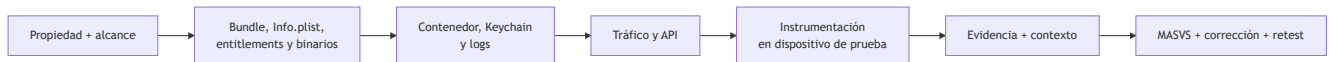
📖 Temas

#	Tema	Por qué importa
1	Preparación del dispositivo jailbroken	Base para el análisis dinámico
2	Descifrado del IPA (FairPlay)	El binario de App Store está cifrado
3	Análisis estático de Mach-O	Revela lógica, cadenas y entitlements
4	Almacenamiento y Keychain	Fugas de datos sensibles frecuentes
5	Interceptación TLS y pinning	Ver el tráfico real de la app
6	Detección de jailbreak y bypass	Habilita el análisis dinámico
7	Reporte MASVS	Estandariza los hallazgos

Explicación en profundidad

El plan de prueba debe sobrevivir a las limitaciones del entorno

Una evaluación iOS combina revisión de IPA propia o autorizada, configuración, almacenamiento, comunicaciones, autenticación, IPC y lógica de negocio. Un dispositivo con jailbreak facilita inspección e instrumentación, pero cambia el entorno y puede desactivar supuestos de plataforma; el resultado debe indicar qué dependió de esa condición.



El **Info.plist** y los entitlements muestran esquemas, dominios asociados, transport security, grupos y capacidades. Strings y símbolos orientan la revisión, pero Swift y optimización dificultan reconstruir intención. En datos se revisan archivos, bases, snapshots, pasteboard, logs y backups; hallar una cadena sensible requiere demostrar persistencia, accesibilidad y consecuencia.

En red se usan cuentas de laboratorio y backend incluido en alcance. Instrumentar pinning permite observar el protocolo, no autoriza atacar endpoints de terceros. La autorización se prueba con identidades y objetos distintos. Controles locales como jailbreak detection o flags de UI se evalúan como defensa adicional; la decisión de negocio permanece en servidor.

Caso razonado: URL scheme de pago

Una app registra un esquema que recibe monto y beneficiario. Otra app de laboratorio puede invocarlo, pero la operación exige sesión y confirmación contextual; no hay transferencia automática. El hallazgo no es «scheme explotable» por existencia: se documenta superficie, se prueba validación y se recomienda universal link y firma contextual si reduce riesgo.



Glosario operativo

Término	Definición útil
IPA	Paquete distribuible que contiene bundle y binarios de una app iOS.
Jailbreak	Modificación que amplía acceso; altera el modelo del dispositivo de prueba.
Info.plist	Configuración declarativa del bundle y varias superficies.
Universal link	Enlace web asociado criptográficamente con una aplicación y dominio.
Snapshot	Imagen de interfaz que puede persistir al pasar a segundo plano.



Criterio de dominio

Hay dominio cuando el alumno declara supuestos del dispositivo, vincula evidencia de bundle, datos, red y runtime con una propiedad, evita atacar terceros y produce un retest reproducible.

Definiciones y características

- **frida-ios-dump**: herramienta que vuelca el binario descifrado desde memoria en un dispositivo con jailbreak. Característica: sorte el cifrado FairPlay para permitir RE.
- **Info.plist**: manifiesto de la app con configuración, permisos de privacidad y ATS. Característica: revela endpoints y excepciones de seguridad de transporte.
- **App Transport Security (ATS)**: política que exige HTTPS moderno; sus excepciones (`NSAllowsArbitraryLoads`) debilitan la comunicación. Característica: hallazgo común de MASVS-NETWORK.
- **objection**: wrapper de Frida con comandos iOS (`ios keychain dump` , `ios sslpinning disable` , `ios jailbreak disable`). Característica: automatiza tareas típicas.
- **Cycript/Frida**: instrumentación en runtime para hookear métodos Objective-C/Swift. Característica: permite modificar el comportamiento en vivo.
- **Data Protection class**: clase de accesibilidad de un ítem del Keychain. Característica: determina si un dato es legible con el dispositivo bloqueado.

Herramientas y preparación

```
# Dispositivo propio jailbroken con OpenSSH y Frida
pip install frida-tools objection
frida-ps -Uai                                # listar apps instaladas por Frida (USB)

# Descifrar una app propia
git clone https://github.com/AloneMonkey/frida-ios-dump
python3 dump.py <nombre_app>                 # genera un .ipa descifrado
```

- **Burp Suite/mitmproxy** con su CA instalado como perfil de confianza en iOS.
- **class-dump**, **Hopper/Ghidra**, **plutil** para inspeccionar plists.

Laboratorio guiado

1. **Instala la app vulnerable** propia (DVIA-v2) en el dispositivo jailbroken.
2. **Descifra el binario**: ejecuta `python3 dump.py DVIA-v2` y obtén el IPA descifrado.
3. **Analiza estáticamente**: descomprime el IPA; revisa `Info.plist` con `plutil -p`, extrae entitlements (`codesign -d --entitlements :-`) y vuelca clases con `class-dump`.
4. **Busca secretos**: `strings <binario> | grep -iE "http://|apikey|secret|token"`.
5. **Inspecciona almacenamiento**: con objection, `env` para localizar directorios, luego revisa `Documents/` , `Library/Preferences/*.plist` y bases SQLite.
6. **Vuelca el Keychain**: `objection -g <app> explore` → `ios keychain dump` y anota clases de accesibilidad.
7. **Intercepta TLS**: configura proxy y CA; si hay pinning, ejecuta `ios sslpinning disable` . Verifica el tráfico en Burp.
8. **Salta jailbreak detection**: si la app se cierra, usa `ios jailbreak disable` o un script Frida, y documenta el bypass.
9. **Reporta** cada hallazgo con requisito MASVS, evidencia y remediación.

Ejercicios

1. Descifra una app propia y confirma con `otool -l` que la sección `LC_ENCRYPTION_INFO` quedó a 0.
2. Localiza en el `Info.plist` cualquier excepción ATS y evalúa su riesgo.
3. Vuelca el Keychain de una app y clasifica cada ítem por su clase de protección.
4. Escribe un script Frida que hooke un método de detección de jailbreak y lo neutralice.
5. Compara el tráfico antes y después de evadir el pinning.
6. Mapea tus hallazgos a MASVS-STORAGE, MASVS-NETWORK y MASVS-RESILIENCE.

Reto verificable

Completa un pentest de una app iOS vulnerable y entrega evidencia de **tres hallazgos** de distinta categoría. **Criterio de aceptación:** demuestras interceptación exitosa del tráfico HTTPS tras evadir el pinning con una captura, e incluyes el volcado del Keychain señalando al menos un ítem con clase de accesibilidad demasiado permisiva.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>dump.py</code> falla al conectar	frida-server no corre en el dispositivo o versión distinta; alinea versiones
class-dump devuelve poco	App en Swift; usa Frida/Ghidra en su lugar
Burp sin tráfico	CA no confiado o pinning; confía el perfil y evade pinning
App se cierra al abrir	Jailbreak detection; usa objection para desactivarla
<code>codesign</code> no muestra entitlements	Binario cifrado; descífralo primero

Preguntas frecuentes

? ¿Por qué hay que descifrar el binario antes de analizarlo? Las apps de App Store se distribuyen con cifrado FairPlay; el RE estático solo es útil sobre el binario descifrado que se vuelca desde memoria.

? ¿Es más difícil hookear Swift que Objective-C? Sí: Swift no expone metadatos de runtime tan ricos como Objective-C, por lo que class-dump aporta menos y suele recurrirse a Frida y análisis del binario.

? ¿Puedo hacer el pentest en el simulador de Xcode? Parcialmente: el simulador usa binarios x86/arm no cifrados y facilita ciertas pruebas, pero no reproduce el Secure Enclave ni el comportamiento real del hardware.

Referencias

- OWASP MASTG — iOS testing: <https://mas.owasp.org/MASTG/>
- frida-ios-dump: <https://github.com/AloneMonkey/frida-ios-dump>
- DVIA-v2: <https://github.com/prateek147/DVIA-v2>
- *The Mobile Application Hacker's Handbook*, caps. de análisis iOS.



Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.



Clase anterior

[Clase 263 — Seguridad de iOS: arquitectura](#)



Siguiente clase

[Clase 265 — Ingeniería inversa de aplicaciones móviles](#)